



Aprendizaje por Refuerzo

Práctica 1: Problema del Bandido de k -Brazos

Máster en Inteligencia Artificial
Universidad de Murcia

Índice

1. Definición del Problema	1
2. Programa Inicial	1
3. Tareas a Realizar	1
3.1. Añadir más gráficos	1
3.2. Implementación de Algoritmos Explicados en Clase	2
4. Entrega	3

“La curiosidad es la esencia de nuestra existencia.”

Richard Feynman (físico)^a

^aEn el aprendizaje por refuerzo los agentes deben equilibrar la exploración (probar nuevas acciones) y la explotación (usar conocimiento previo).

Resumen

En este trabajo se analizan los algoritmos del bandido de k -brazos, comparando su rendimiento en entornos estacionarios. Para ello se implementan varias familias de métodos: ϵ -greedy, UCB, ascenso del gradiente y métodos bayesianos, evaluando las recompensas y rechazos acumulados.

1. Definición del Problema

El **problema del bandido de k -brazos** modela una situación de toma de decisiones secuencial bajo incertidumbre. Consiste en lo siguiente:

- ▶ Una máquina tragaperras con k brazos (acciones)
- ▶ Cada brazo i tiene una distribución de recompensa desconocida $\mathcal{P}_i$
- ▶ En cada paso t , el agente:
 1. Elige un brazo $a_t \in \{1, \dots, k\}$
 2. Recibe una recompensa $r_t \sim \mathcal{P}_{a_t}$
- ▶ El objetivo es maximizar la **recompensa acumulada** $\sum_{t=1}^T r_t$ donde T es el horizonte temporal.
- ▶ Una política es tomar decisiones que tengan en cuenta aquellos brazos que, por la experiencia, han mostrado tener mayores recompensas.
- ▶ Alternativamente, otra forma de medir las mejores decisiones es considerar como medida de desempeño el **rechazo** (*regret*) que cuantifica la pérdida por no elegir siempre el mejor brazo:

$$R(T) = T\mu^* - \mathbb{E} \left[\sum_{t=1}^T \mu_{a_t} \right]$$

donde $\mu^* = \max_i \mu_i$ y $\mu_i = \mathbb{E}[\mathcal{P}_i]$.

- ▶ La consideración del rechazo da lugar al estudio de una multitud de algoritmos alternativos.

El problema del bandido multibrazo es el mejor ejemplo del dilema de la **Exploración-Explotación**. Los mejores algoritmos son los que son capaces de encontrar el mejor equilibrio entre ambas opciones.

2. Programa Inicial

Usando https://github.com/ldaniel-hm/eml_k_bandit/blob/main/bandit_experiment.ipynb trabajaremos con el repositorio https://github.com/ldaniel-hm/eml_k_bandit.

Es un ejemplo de cómo se pueden estudiar los algoritmos objeto de estudio. En este caso se comparan tres algoritmos ϵ -greedy.

3. Tareas a Realizar

El código del repositorio y del notebook inicial están incompletos: queremos hacer un estudio más completo sobre este conjunto de técnicas.

3.1. Añadir más gráficos

Se consideran *necesarias* las siguientes gráficas:

- ▶ Mostrar el porcentaje en el que fue seleccionado el brazo óptimo.

```
def plot_optimal_selections(steps: int,  
                             optimal_selections: np.ndarray,  
                             algorithms: List[Algorithm]):  
    """
```

Genera la gráfica de Porcentaje de Selección del Brazo Óptimo vs Pasos de Tiempo.

```
:param steps: Número de pasos de tiempo.
:param optimal_selections: Matriz de porcentaje de selecciones óptimas.
:param algorithms: Lista de instancias de algoritmos comparados.
"""
```

- Mostrar las estadísticas de cada brazo. Cada valor en el eje X representará un brazo. En el eje Y se representa el promedio de las ganancias de cada brazo. El gráfico mostrará un histograma para cada brazo donde se muestre el promedio de las ganancias obtenidas pero en el eje X, como etiqueta, se mostrará también el número de veces que fue seleccionado el brazo e indicar si es el brazo óptimo o no.

Añade lo que considere necesario que ayude a clarificar el significado de la gráfica.

```
def plot_arm_statistics(arm_stats: LoQueConsideres,
                      algorithms: List[Algorithm], *args):
    """
    Genera gráficas separadas de Selección de Arms:
    Ganancias vs Pérdidas para cada algoritmo.
    :param arm_stats: Lista (de diccionarios) con estadísticas de cada brazo por algoritmo.
    :param algorithms: Lista de instancias de algoritmos comparados.
    :param args: Parámetros que consideres
    """
```

- Mostrar la evolución del rechazo, de forma análoga a cómo se muestra la evolución de las recompensas.

```
def plot_regret(steps: int,
               regret_accumulated: np.ndarray,
               algorithms: List[Algorithm], *args):
    """
    Genera la gráfica de Regret Acumulado vs Pasos de Tiempo
    :param steps: Número de pasos de tiempo.
    :param regret_accumulated: Matriz de regret acumulado (algoritmos x pasos).
    :param algorithms: Lista de instancias de algoritmos comparados.
    :param args: Parámetros que consideres. P.e. la cota teórica  $Cte * \ln(T)$ .
    """
```

Qué debe de hacer

- Modifique los ficheros https://github.com/ldaniel-hm/eml_k_bandit/blob/main/bandit_experiment.ipynb y https://github.com/ldaniel-hm/eml_k_bandit/blob/main/plotting/plotting.py para incluir las gráficas indicadas.

Se pueden modificar los parámetros de las funciones de dibujo si se viera conveniente (aquí solo se da una propuesta) mientras que cumpla con su objetivo. Obviamente, también se modificará el resto del código si se viera necesario.

- Complete el estudio de la familia de algoritmos ϵ -greedy e indique qué gráficas son las más relevantes ¿por qué?

3.2. Implementación de Algoritmos Explicados en Clase

1. Considerar los siguiente tipos de bandidos.

- Distribución Bernoulli. Cada brazo solo puede dar dos resultados: éxito, 1, o fracaso, 0.
Ejemplo: Un usuario hace clic o no en un anuncio publicitario (Click-Through Rate).

- Distribución Binomial, $\mathcal{B}(n, p)$. Es cuando nos referimos a que, si lo tiramos n veces, el **número total de éxitos** (recompensas positivas) se dará con probabilidad p . La distribución Bernoulli es un caso particular de la binomial, para $n = 1$.

Ejemplo: Publicidad Online. Supongamos que un anunciante quiere optimizar su campaña de publicidad en línea eligiendo entre $k = 3$ diferentes anuncios. Cada anuncio

tiene una probabilidad desconocida de que un usuario haga clic en él (CTR - Click-Through Rate). A_1 tiene un CTR desconocido de p_1 , A_2 tiene un CTR desconocido de p_2 y A_3 tiene un CTR desconocido de p_3 .

El anunciante usa un **modelo de bandido de k -brazos**, donde cada vez que muestra un anuncio (elige un brazo), observa si el usuario hizo clic (éxito = 1) o no (fracaso = 0). Después de mostrar (explorar) el anuncio A_j un total de n_j veces y recibir (aprender) X_j clics, el número de éxitos sigue una distribución binomial: $X_j \sim \text{Bin}(n_j, p_j)$.

Entonces se mostrará (explotar) el anuncio que tiene mayor p_j .

- ▶ Distribución Normal, $\mathcal{N}(\mu, \sigma^2)$. Si $X \sim \mathcal{B}(n, p)$ con $np \geq 5$ y $n(1 - p) \geq 5$ se garantiza que la distribución binomial no está demasiado sesgada y tenga una forma aproximadamente simétrica. Entonces se puede hacer una **aproximación normal**: $X \approx \mathcal{N}(\mu, \sigma^2)$ donde: $\mu = np$ y $\sigma^2 = np(1 - p)$.
- ▶ Distribución Beta, $\mathcal{B}(\alpha, \beta)$. Se usa principalmente en el algoritmo de muestreo de Thompson (aprendizaje bayesiano) partiendo de una distribución $\mathcal{B}(1, 1)$ para cada brazo.

2. Realice los estudios comparativos entre algoritmos del mismo grupo.

- ▶ Métodos Greedy: Son los del apartado anterior.
- ▶ Métodos UCB: UCB1 y UCB2.
- ▶ Métodos de Ascenso del Gradiente: Softmax y Gradiente de Preferencias.
- ▶ Métodos Bayesianos: Muestreo de Thompson y UCB Bayesiano.

No tiene por qué mostrar los 4 gráficos del primer estudio, puede que no le aporte nada tener tantos gráficos. Por eso se pide cuál de ellos son los importantes.

3. Realice los estudios que considere entre algoritmos de distintos grupos.

Por ejemplo: $\epsilon = 0,1$ -greedy frente a UCB1, Softmax y muestreo de Thompson.

! Importante

- ▶ **Para cada estudio** construya un notebook diferente.
- ▶ Utilice siempre la misma semilla para la generación de números aleatorios para que sus gráficas sean reproducibles.
- ▶ (Opcional) Intente, si tiene suerte, usar semillas donde una técnica se vea favorecida y semillas donde la técnica no obtenga tan buenos resultados. Esto le permitiría ampliar el estudio a todos estos casos.

4. Entrega

El trabajo se entregará vía **GitHub** y debe seguir las siguientes instrucciones:

1. Crear un repositorio llamado **k_brazos_X**, donde **X** es el nombre del grupo. Por ejemplo, las iniciales de los apellidos de sus miembros. El nombre no tendrá más de 8 caracteres.
2. Incluir los **notebooks ya resueltos** en el repositorio.
3. Incluir el fichero README.md con los siguientes datos.

```
# Título del Trabajo

## Información
- **Alumnos:** Apellido1, Nombre1; Apellido2, Nombre2; ...
- **Asignatura:** Nombre de la asignatura
- **Curso:** 2024/2025
- **Grupo:** X

## Descripción
[Breve descripción del trabajo y sus objetivos]

## Estructura
[Explicación de la organización del repositorio]
```

```
## Instalación y Uso  
[Instrucciones si son necesarias]
```

```
## Tecnologías Utilizadas  
[Lista de lenguajes, frameworks, etc.]
```

Se sugiere la siguiente estructura general, con los comentarios incluidos.

```
k_brazos_X/  
|-- src/                # Código .py (expanda src, o considere varios src)  
|-- docs/              # Documentación ( .bib, .pdf, ... si procede)  
|-- tests/             # Tests (si aplica)  
|-- data/              # Datos necesarios (si aplica)  
|-- README.md  
|-- inicio-k_brazos_X.ipynb # Breve introducción del problema y enlace a los estudios.  
|-- epsilon_greedy.ipynb   # Estudio de la familia epsilon-greedy  
|-- estudio_x.ipynb       # Poner tantos como estudios realizados (al menos 5)
```

4. Todo se ejecutará en Colab. A partir del enlace

https://colab.research.google.com/github/autores/k_brazos_X/blob/main/inicio-k_brazos_X.ipynb

las primeras celdas harán una copia del repositorio y se instalarán los paquetes que sean necesarios. A partir de dicho *notebook* se podrá navegar entre los distintos *notebooks*.

! Importante

El **prerequisito** para la evaluación es que al entrar por primera vez en el *notebook* y seleccionar la opción Entorno de ejecución > Ejecutar todas **no se produzcan errores** y el resultado final sea exactamente el mismo al que se encuentra en el repositorio. Es decir, que sus notebooks **son reproducibles**.

- ▶ Tendrán que garantizar que las celdas iniciales de cada *notebook* realicen todo lo necesario para que no se produzca error alguno en su ejecución.
- ▶ La no-ejecución completa de un notebook, conlleva que todo su contenido quedará descartado y no se considerará superado.